

Comparative Performance of CNN From Scratch and Fine-Tuned MobileNetV2 for Tomato Leaf Disease Classification

Gusli Yanza¹

¹ Ilmu Komputer, System Komputer, Universtas Dinamika Bangsa, Jambi, Indonesia

Email: gusliyanza02@gmail.com

Corresponding Author Email : gusliyanza02@gmail.com

Submitted : **Abstrak–Background:** Tomato leaf diseases can reduce crop quality and productivity, making accurate image-based identification important for early disease management. Objective: This study compares a convolutional neural network developed from scratch with a fine-tuned MobileNetV2 model for multiclass tomato leaf disease classification. Methods: The PlantVillage tomato dataset containing 14,521 images from 10 classes was divided into training, validation, and testing sets using an 80:10:10 ratio. Images were resized to 224 × 224 pixels, and data augmentation was applied to the training set. Both models were evaluated using accuracy, precision, recall, F1-score, classification reports, and confusion matrices. Results: The CNN from scratch achieved a testing accuracy of 98.90%, with macro precision, recall, and F1-score of approximately 0.99. The fine-tuned MobileNetV2 achieved a testing accuracy of 94.77%, with macro precision of 0.94 and macro recall and F1-score of 0.93. Confusion matrix analysis showed that the custom CNN produced fewer misclassifications, particularly across visually similar disease classes. Conclusion: The CNN from scratch provided the best performance for this dataset, showing that transfer learning does not always outperform a custom architecture when sufficient domain-specific and visually consistent training data are available.

Revision :

Accepted:

Published:

Keywords: Convolutional Neural Network, Deep Learning, MobileNetV2, PlantVillage, Tomato Leaf Disease, Image Classification

1. INTRODUCTION

Plant diseases constitute a major challenge in agricultural production because they can reduce crop quality, lower productivity, and increase the cost of disease control. Tomato plants are susceptible to various diseases caused by fungi, bacteria, viruses, and pests. Many of these diseases produce visible symptoms on the leaves, including spots, discoloration, mold, curling, tissue damage, and irregular surface patterns. Early recognition of these symptoms is important because an incorrect or delayed diagnosis may allow the infection to spread to other parts of the plant or to surrounding crops. Conventional identification commonly depends on direct visual inspection by farmers or agricultural specialists. However, manual inspection is time-consuming, subjective, and highly dependent on the experience of the observer, particularly when different diseases exhibit similar visual symptoms [1].

The development of artificial intelligence has introduced computer vision as an alternative method for automatically examining plant images. In image classification, a model receives an image and assigns it to one of several predetermined categories. For tomato leaf disease recognition, the categories may represent healthy leaves or diseases such as bacterial spot, early blight, late blight, leaf mold, Septoria leaf spot, target spot, tomato mosaic virus, tomato yellow leaf curl virus, and two-spotted spider mite damage. Automated classification can provide more consistent predictions than unaided visual inspection and may support faster decision-making in disease management. A lightweight MobileNetV2-based system developed for tomato leaves demonstrated that accurate classification can be achieved while maintaining a relatively small model size, making image classification relevant not only for laboratory analysis but also for mobile and embedded applications [2].

Deep learning has become one of the most widely used approaches for image-based plant disease classification. Unlike traditional machine-learning methods, which generally require manual feature extraction, deep neural networks can automatically learn visual representations from raw images. A study that processed plant-leaf images directly in the JPEG compressed domain showed that transfer learning could be applied while reducing the need for complete image-domain reconstruction, demonstrating the flexibility of deep learning for efficient plant disease recognition [3]. Among deep-learning techniques, Convolutional Neural Networks (CNNs) are particularly effective because convolutional layers can detect local visual patterns such as edges, textures, color transitions, lesions, and disease spots. Features extracted in early layers are progressively combined into more complex representations in deeper layers, enabling a model to distinguish between visually similar disease categories.

Recent research has investigated increasingly advanced architectures for plant disease identification. Thakur et al. proposed PlantXViT, an explainable architecture combining convolutional operations and Vision Transformer components. The architecture was evaluated on several plant-disease datasets and demonstrated that CNN-based local feature extraction could be combined with global attention mechanisms while maintaining a relatively lightweight structure [4]. However, transformer-based architectures may require more complicated implementation and computational resources than conventional sequential CNNs, particularly for student projects or deployment on resource-limited devices.

The availability and balance of labeled images are also important considerations. Thuseethan et al. proposed a lightweight Siamese-network framework for tomato leaf disease recognition. Their method achieved competitive performance on PlantVillage and another tomato dataset while using approximately 2.96 million trainable parameters. The study demonstrated that Siamese learning can address limited and imbalanced training data [5]. Nevertheless, Siamese networks use paired-image training and a more specialized learning procedure, whereas standard multiclass CNN classification is generally easier to develop, train, interpret, and deploy.

Transfer learning represents another major strategy in plant disease classification. Instead of training all convolutional filters from random initialization, transfer learning uses a network that has previously learned general visual features from a large-scale dataset. Ashmafee et al. applied an EfficientNetV2S-based transfer-learning pipeline for apple leaf disease classification and incorporated runtime augmentation to address class imbalance. The results indicated that pretrained networks can provide strong performance when their learned representations are adapted to a plant-specific classification task [6]. Nevertheless, results obtained for one crop or dataset cannot be assumed to apply directly to another because each dataset has different disease symptoms, class distributions, backgrounds, lighting conditions, and image-acquisition procedures.

Yao et al. compared multiple CNN backbones and learning strategies across PlantVillage, Plant Leaves, and PlantDoc. Their experiments showed that model performance depended on both the selected architecture and dataset characteristics. InceptionV3 performed strongly in their multi-prediction framework, while the findings also indicated that a carefully designed single model could compete with more complicated multi-model solutions [7]. This evidence supports the need to evaluate architectures under the same experimental conditions rather than assuming that a larger or more complex network will always produce better results.

Comparative experiments conducted by Yasin and Fatima further demonstrated substantial differences among CNN, MobileNet, EfficientNetB0, Xception, DenseNet, and Inception-based architectures for tomato and corn leaf images. Their results varied according to the dataset composition and experimental configuration, showing that model selection has a direct effect on classification accuracy and validation behavior [8]. Therefore, an appropriate comparison should use an identical dataset split, image resolution, augmentation procedure, optimization configuration, and testing set so that performance differences can be attributed more fairly to the selected architecture.

Deployment requirements have also influenced recent plant disease research. Jha et al. combined attention-based visual feature extraction, Green Chromatic Coordinate features, and a Support Vector Machine classifier. They also investigated model quantization for integration with Internet of Things devices and reported that reducing model size could preserve useful classification performance [9]. Such findings emphasize that model evaluation should not be limited to accuracy. Parameter count, training time, model size, inference efficiency, and compatibility with mobile or browser-based environments should also be considered.

Although high classification accuracy has been widely reported, model transparency remains a significant concern. Amara et al. applied Testing with Concept Activation Vectors to analyze whether plant-disease classifiers relied on meaningful visual concepts such as color, texture, and disease-related patterns [10]. Their work highlighted that a model may produce a correct class prediction without necessarily using medically or agriculturally meaningful evidence. Explainability is important because farmers and agricultural specialists may be reluctant to trust an automated diagnosis when the model provides no indication of the visual evidence underlying its prediction.

The effectiveness of CNNs has also been observed in disease classification for other crops. Tambe et al. developed a CNN for potato leaf disease classification involving early blight, late blight, and healthy leaves. Their results showed that a CNN trained directly on a crop-specific dataset could achieve high classification performance [11]. This supports the possibility that a CNN trained from scratch may perform strongly when the target dataset contains sufficient and visually consistent images.

Ouamane et al. proposed a tomato disease classification approach that combined CNN feature extraction with Higher-Order Whitenized Singular Value Decomposition and Multilinear Discriminant Analysis. Their method achieved 98.36% accuracy on PlantVillage but 89.39% on a Taiwan tomato dataset [12]. The difference between these results illustrates an important generalization issue: strong performance on controlled images does not necessarily translate into equivalent performance on images captured under different environmental conditions.

A related explainability study using Automated Concept-based Explanation found that plant-disease models could learn both useful disease patterns and incidental characteristics related to backgrounds or illumination [13]. This issue is particularly important for PlantVillage because many of its images contain centered leaves and relatively consistent backgrounds. Models trained and tested exclusively on this type of data may partly learn dataset-specific characteristics instead of relying only on disease symptoms.

Data augmentation is commonly used to reduce overfitting and increase variation during training. Ferdi investigated background removal as an augmentation strategy for MobileNetV2-based apple leaf disease classification. The study reported improved performance when the training data were expanded with background-removed images [14]. This finding indicates that augmentation should preserve important disease characteristics while reducing excessive dependence on irrelevant background information. Rotation, horizontal flipping, zooming, shifting, and other transformations can help a model learn more robust features, but unrealistic augmentation may distort lesions or leaf structures.

Other studies have compared established transfer-learning architectures. Hosen and Islam evaluated VGG19 and InceptionV3 for tomato leaf disease recognition and demonstrated that pretrained feature extractors could identify multiple tomato leaf conditions [15]. However, the reported performance remained dependent on normalization, resizing, dropout, augmentation, the classification head, and the training configuration. Therefore, transfer learning does not automatically guarantee superior accuracy.

A comparison involving CNN, Vision Transformer, and conventional machine-learning architectures for rice leaf disease classification similarly found that performance differed among model families, with ResNet50 providing the strongest result under the study's configuration [16]. In another multi-crop experiment, Kanakala and Ningappa compared CNN and Long Short-Term Memory models and found that the CNN achieved better validation accuracy for image-based leaf disease classification [17]. Collectively, these studies demonstrate that architecture effectiveness is task-dependent and must be established through controlled empirical evaluation.

MobileNetV2 was selected as the pretrained model in the present study because it was specifically designed for efficient image recognition using inverted residual blocks, linear bottlenecks, and depthwise separable convolutions. These mechanisms reduce computational cost and parameter requirements compared with many conventional deep CNN architectures [18]. Its relatively compact design also makes it suitable for conversion to TensorFlow Lite and deployment on mobile or embedded systems.

Despite extensive research on plant disease classification, several limitations remain. Many studies evaluate only one proposed model, use different dataset partitions, or report accuracy without a direct comparison under identical experimental conditions. Other studies focus on sophisticated architectures that may be difficult to reproduce or deploy. Furthermore, transfer learning is frequently assumed to be superior to a custom CNN, although a pretrained network learns general features from its original dataset, whereas a model developed from scratch learns representations directly from the target domain. When the target dataset is sufficiently large and visually consistent, a custom CNN may equal or outperform a pretrained network.

Accordingly, the research gap addressed in this study is the lack of a controlled comparison between a sequential CNN developed from scratch and a fine-tuned MobileNetV2 using the same tomato leaf dataset, preprocessing procedure, image resolution, data split, augmentation strategy, and evaluation metrics. The study uses 14,521 PlantVillage tomato leaf images distributed across 10 classes. The images are divided into training, validation, and testing sets using an 80:10:10 ratio and resized to 224×224 pixels.

The objective of this study is to develop and compare CNN from scratch and MobileNetV2 transfer-learning models for multiclass tomato leaf disease classification. The models are evaluated using training accuracy, validation accuracy, testing accuracy, precision, recall, F1-score, confusion matrix, number of parameters, training duration, and model file size. The study also implements feature extraction and fine-tuning stages for MobileNetV2 and converts the selected model into TensorFlow SavedModel, TensorFlow Lite, and TensorFlow.js formats. The expected contribution is an empirical explanation of whether a custom CNN or a lightweight pretrained model provides the most effective balance between classification performance, computational complexity, and deployment readiness for the selected dataset.

2. RESEARCH METHOD

2.1 Research Stages (Subtitle Times New Roman 10, Left, After 6 pt)

This study employed a quantitative experimental approach to develop and compare two deep-learning models for multiclass tomato leaf disease classification. The first model was a Convolutional Neural Network developed from scratch using the Keras Sequential architecture, while the second model employed MobileNetV2 transfer learning with ImageNet initialization.

The research process consisted of dataset acquisition, dataset validation and organization, exploratory analysis, image preprocessing, data augmentation, model development, model training, performance evaluation, model comparison, model conversion, and image inference. The complete research stages are illustrated in Figure 1.

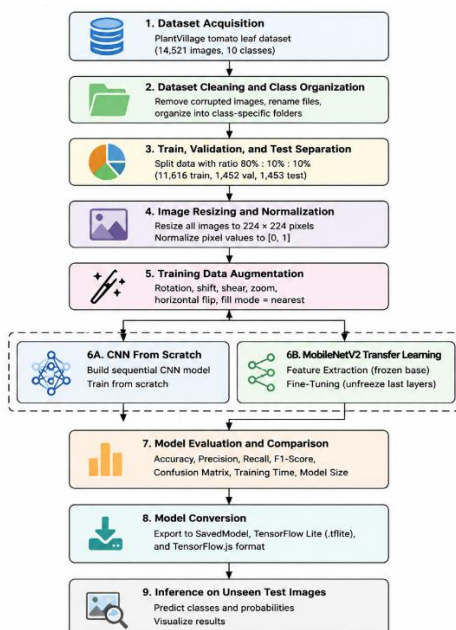


Figure 1. Research stages of tomato leaf disease classification

The dataset was first organized into class-specific directories. The images were then separated into training, validation, and testing subsets. The same data partitions and input dimensions were applied to both models to ensure a consistent and fair comparison. Model evaluation was conducted using accuracy, precision, recall, F1-score, classification reports, and confusion matrices.

2.2 Dataset Distribution (After 6 pt, Before 6 pt)

The dataset used in this study was the tomato leaf subset of the PlantVillage dataset. It was obtained from a Kaggle repository and contained images of healthy tomato leaves and leaves affected by nine disease or pest categories. The dataset consisted of 14,521 images distributed across 10 classes.

The classes included bacterial spot, early blight, healthy leaf, late blight, leaf mold, Septoria leaf spot, target spot, tomato mosaic virus, tomato yellow leaf curl virus, and two-spotted spider mite. Each image was stored in a directory corresponding to its class label, allowing TensorFlow to automatically assign categorical labels based on the folder names.

The dataset was divided into training, validation, and testing subsets using an approximately 80:10:10 proportion. The training subset contained 11,616 images, the validation subset contained 1,452 images, and the testing subset contained 1,453 images. This proportion was selected because a large training subset was required to learn complex disease characteristics, while the validation and testing subsets remained sufficiently large for hyperparameter monitoring and unbiased model evaluation.

Table 1. Distribution of the PlantVillage tomato leaf dataset

Class	Training	Validation	Testing	Total
Bacterial spot	1,361	170	171	1,702
Early blight	640	80	80	800
Healthy	1,014	127	126	1,267
Late blight	1,218	152	153	1,523
Leaf mold	610	76	76	762
Septoria leaf spot	1,133	142	142	1,417
Target spot	899	113	112	1,124
Tomato mosaic virus	239	30	30	299
Tomato yellow leaf curl virus	3,429	428	429	4,286
Two-spotted spider mite	1,073	138	134	1,341
Total	11,616	1,452	1,453	14,521

The distribution showed that the dataset was not completely balanced. The tomato yellow leaf curl virus class contained the largest number of images, while the tomato mosaic virus class contained the smallest number. Therefore, evaluation was not limited to overall accuracy but also incorporated macro-averaged precision, recall, and F1-score to provide equal consideration to every class.

2.3 Data Preparation

All images were resized to 224×224 pixels with three RGB channels. This image size was selected because it is compatible with MobileNetV2 and provides sufficient spatial information for identifying leaf texture, discoloration, lesions, and disease patterns. A batch size of 32 was used for all training, validation, and testing processes. A random seed of 42 was applied to improve experimental reproducibility.

Pixel values were normalized from the original range of 0–255 to the range of 0–1 using a rescaling factor of $1/255$. The training data generator shuffled the images during each training epoch, whereas the validation and testing generators maintained a fixed image order. The fixed order was necessary to ensure that the predicted labels remained aligned with the original class labels when generating classification reports and confusion matrices.

Data augmentation was applied only to the training subset. This procedure generated different variations of the original images during training without permanently creating additional image files. The augmentation configuration included a rotation range of 30 degrees, horizontal and vertical shifts of 20%, a shear range of 15%, a zoom range of 20%, and horizontal flipping. Empty pixels produced by geometric transformations were filled using the nearest-neighbor strategy. Vertical flipping was not applied because an upside-down transformation could produce leaf orientations that were less representative of the original data.

The augmentation operations were expressed as follows:

- Rotation range: 30 degrees;
- Width shift range: 0.20;
- Height shift range: 0.20;
- Shear range: 0.15;
- Zoom range: 0.20;
- Horizontal flip: enabled;
- Vertical flip: disabled;
- Fill mode: nearest.

The validation and testing subsets were not augmented. Only pixel normalization was applied to these subsets so that model performance was measured using unchanged images. Figure 2 presents an example of an original bacterial spot image and several augmented variations.

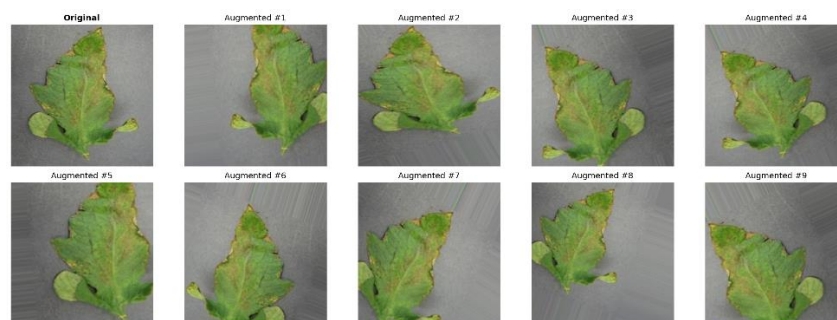


Figure 2. Examples of data augmentation applied to a bacterial spot image

2.4 CNN From Scratch Architecture

The first model was developed from scratch using the Keras Sequential API. The model received an RGB image with a dimension of $224 \times 224 \times 3$. It consisted of four convolutional blocks followed by a classification head. The first convolutional block contained two Conv2D layers with 32 filters, a 3×3 kernel, ReLU activation, and same padding. Each convolutional layer was followed by Batch Normalization. A 2×2 MaxPooling2D layer reduced the spatial dimensions, followed by a Dropout layer with a rate of 0.25.

The second block followed the same structure but used 64 convolutional filters. The third block used two Conv2D layers with 128 filters, Batch Normalization, MaxPooling2D, and a Dropout rate of 0.30. The fourth block used one Conv2D layer with 256 filters, followed by Batch Normalization, MaxPooling2D, and a Dropout rate of 0.30. After convolutional feature extraction, GlobalAveragePooling2D transformed the final feature maps into a one-dimensional feature vector. The classification head contained a Dense layer with 512 neurons and ReLU activation, Batch Normalization, and a Dropout rate of 0.50. This layer was followed by a Dense layer with 256 neurons and ReLU activation and a Dropout rate of 0.40. The output layer contained 10 neurons with the softmax activation function, corresponding to the 10 tomato leaf classes.

The architecture contained 852,522 parameters, consisting of 850,090 trainable parameters and 2,432 non-trainable parameters. The model architecture is illustrated in Figure 3.

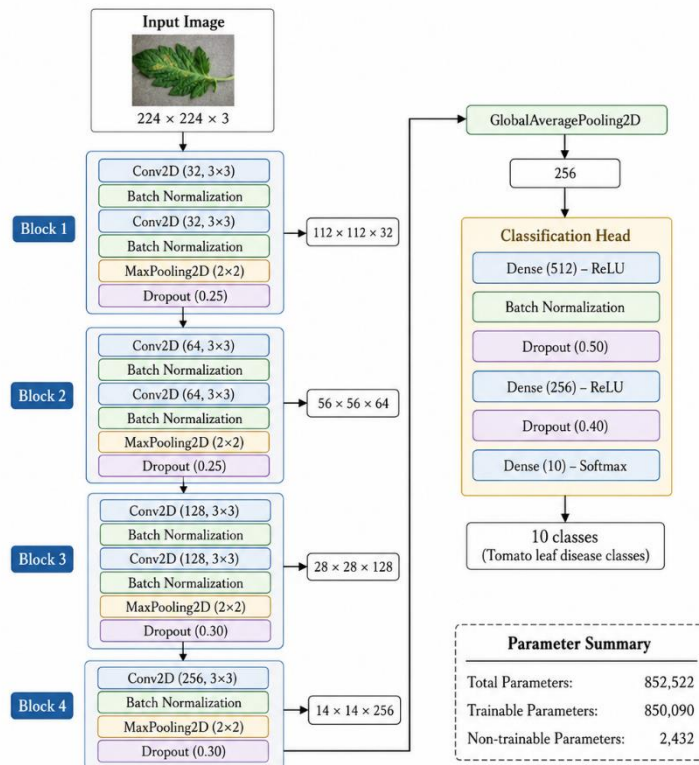


Figure 3. Architecture of the CNN developed from scratch

The model was compiled using the Adam optimizer with an initial learning rate of 0.001. Categorical cross-entropy was used as the loss function because the classification problem involved more than two mutually exclusive classes. The model was trained for a maximum of 50 epochs.

Three callbacks were implemented. EarlyStopping monitored validation loss with a patience of 10 epochs and restored the weights from the best epoch. ReduceLROnPlateau decreased the learning rate by a factor of 0.30 when validation loss did not improve for five epochs, with a minimum learning rate of (10^{-7}) . ModelCheckpoint saved the model that achieved the highest validation accuracy.

2.5 MobileNetV2 Transfer Learning Architecture

The second model used MobileNetV2 initialized with weights learned from the ImageNet dataset. MobileNetV2 was selected because it uses depthwise separable convolutions, inverted residual blocks, and linear bottlenecks to reduce computational requirements while maintaining effective visual feature extraction [18].

The original fully connected classification layer of MobileNetV2 was excluded by setting `include_top=False`. The model received input images with dimensions of $224 \times 224 \times 3$ and generated feature maps with dimensions of $7 \times 7 \times 1,280$.

During the first training stage, all 154 layers in the MobileNetV2 base model were frozen. The pretrained network was used only as a feature extractor. A custom classification head was then added, consisting of GlobalAveragePooling2D, a Dense layer with 256 neurons and ReLU activation, Batch Normalization, Dropout with a rate of 0.40, and a 10-neuron softmax output layer.

The complete model contained 2,589,514 parameters. During feature extraction, 331,018 parameters in the classification head were trainable, while 2,258,496 parameters in the MobileNetV2 base were frozen.

The feature-extraction stage used the Adam optimizer with a learning rate of 0.001 and was trained for a maximum of 30 epochs. EarlyStopping used a patience of eight epochs, while ReduceLROnPlateau reduced the learning rate by a factor of 0.30 after four epochs without validation-loss improvement.

After feature extraction, fine-tuning was performed by unfreezing the final 30 layers of the MobileNetV2 base model. Earlier layers remained frozen because they contained general low-level features such as edges, colors, and textures. The final layers were opened to allow the model to adapt high-level ImageNet features to tomato leaf disease characteristics. This strategy produced 1,857,418 trainable parameters.

Fine-tuning used a smaller learning rate of (10^{-5}) to prevent large weight updates from damaging the pretrained feature representations. The fine-tuning stage was performed for a maximum of 20 epochs.

The transfer-learning architecture is illustrated in Figure 4.

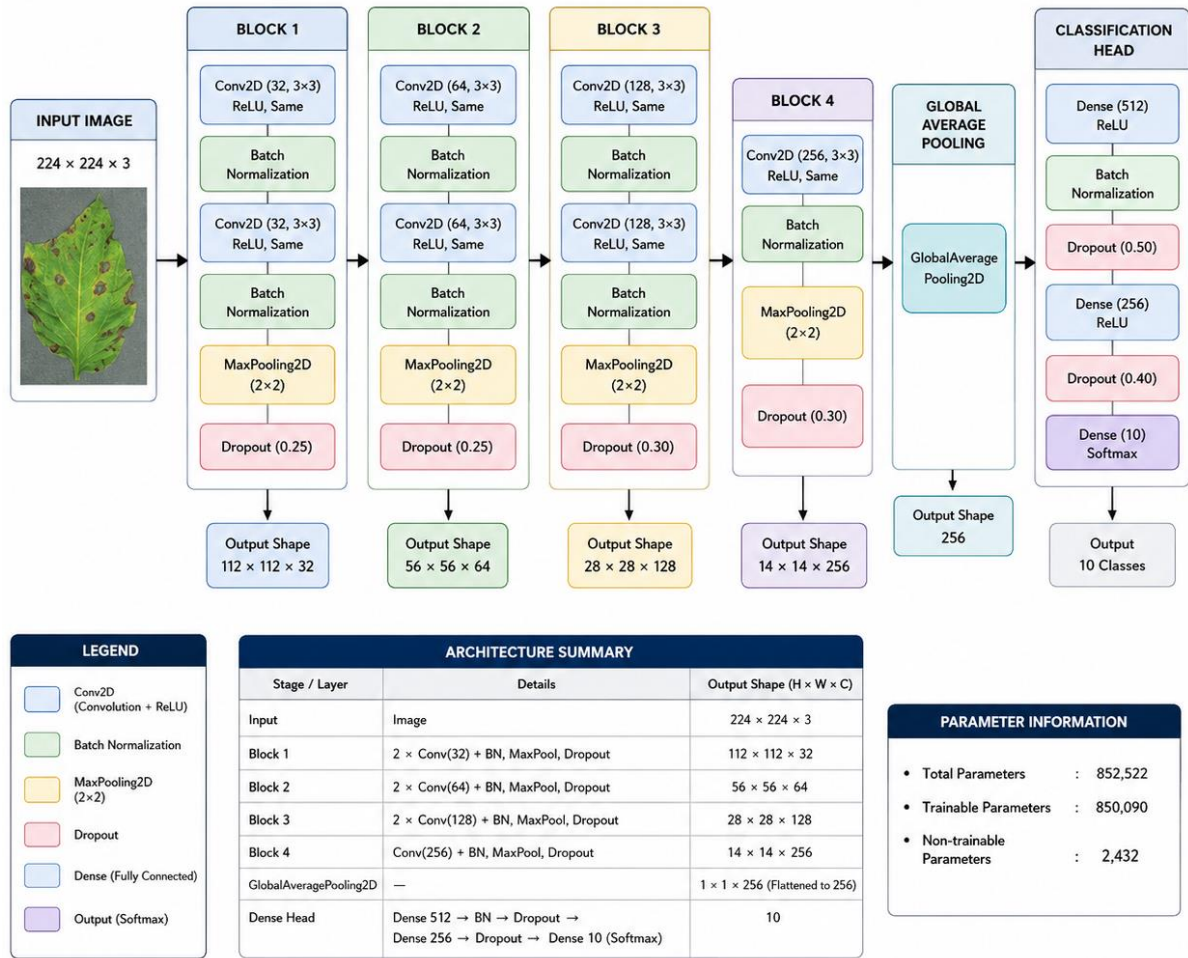


Figure 4. MobileNetV2 feature-extraction and fine-tuning architecture

2.6 Evaluation Metrics

The performance of both models was evaluated using accuracy, precision, recall, F1-score, and confusion matrix. For multiclass evaluation, each class was evaluated using a one-versus-rest interpretation. Accuracy represents the proportion of correctly classified images relative to all evaluated images. It was calculated using Equation (1).

$$Accuracy = \frac{\sum_{i=1}^K \{c_{ii}\}}{N} \quad (1)$$

Precision measures the proportion of images predicted as a particular class that actually belonged to that class. Precision was calculated using Equation (2).

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

Recall measures the proportion of images belonging to a class that were correctly identified by the model. Recall was calculated using Equation (3).

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

The F1-score is the harmonic mean of precision and recall. It provides a balanced measurement when false-positive and false-negative predictions must both be considered. It was calculated using Equation (4).

$$F1\text{-Score} = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (4)$$

Because the dataset contained unequal numbers of images across the 10 classes, macro-averaged metrics were also calculated. The macro average assigns the same weight to each class regardless of its number of samples. For (K) classes, the macro F1-score was calculated using Equation (5).

$$Macro\ F1 = \frac{1}{K} \sum_{i=1}^K F1_i \quad (5)$$

A confusion matrix was used to analyze the relationship between actual and predicted labels. Each matrix element (C_{ij}) represented the number of images from actual class (i) that were predicted as class (j), as expressed in Equation (6).

$$C_{ij} = \sum_{n=1}^N I(y_n=i \wedge \hat{y}_n=j) \quad (6)$$

In these equations, (TP) represents true positives, (TN) represents true negatives, (FP) represents false positives, and (FN) represents false negatives. The symbol (K) represents the number of classes, (N) represents the total number of test images, (y_n) represents the actual label, and ($\hat{y}_n$) represents the predicted label. The indicator function ($I(\cdot)$) returns one when the stated condition is true and zero otherwise.

Categorical cross-entropy was used as the training loss function. For (N) samples and (K) classes, the loss was calculated using Equation (7).

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{p}_{ik}) \quad (7)$$

In Equation (7), (y_{ik}) represents the actual one-hot-encoded label and ($\hat{p}_{ik}$) represents the probability predicted by the model for class (k).

3. RESULTS AND DISCUSSION

This section presents the experimental results obtained from the CNN developed from scratch and the MobileNetV2 transfer-learning model. The analysis covers the training process, testing performance, class-level evaluation, confusion matrices, comparative analysis, model deployment, and inference results. Both models were trained and evaluated using the same PlantVillage tomato leaf dataset, image dimensions, batch size, data partitions, and evaluation procedures to provide a consistent comparison.

3.1 Training Results

3.1.1 CNN From Scratch Training Results

The CNN from scratch was trained for 50 epochs using the Adam optimizer, categorical cross-entropy loss, data augmentation, and several callbacks. The model used an initial learning rate of 0.001. The ReduceLROnPlateau callback reduced the learning rate when the validation loss stopped improving, while ModelCheckpoint stored the model with the highest validation accuracy. EarlyStopping was also configured to restore the weights associated with the lowest validation loss.

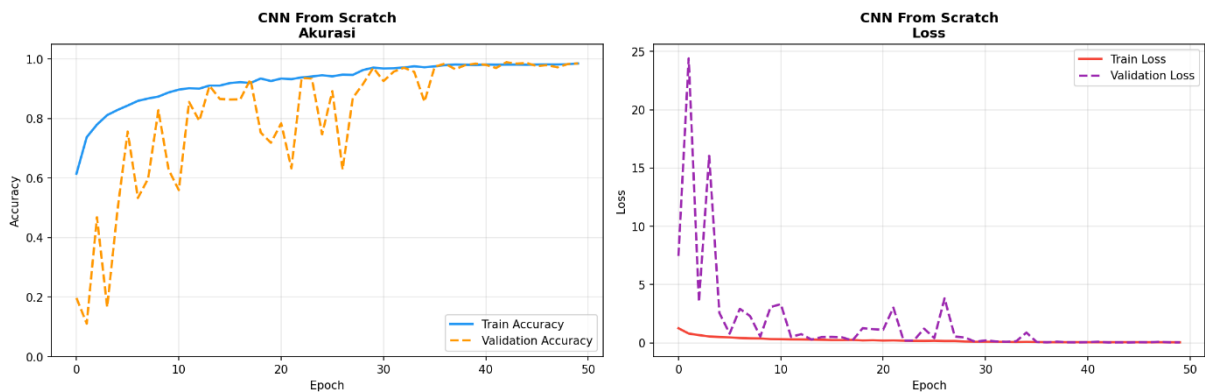


Figure 5. Training and validation accuracy and loss of the CNN from scratch

As shown in Figure 5, the training accuracy increased from approximately 61% during the first epoch to more than 98% at the end of training. The validation accuracy was considerably more unstable during the early epochs. It initially produced several sharp increases and decreases before gradually stabilizing. This fluctuation was also visible in the validation-loss curve, which reached several high values during the first half of the training process. The initial instability may have been influenced by the relatively high learning rate, data augmentation, class imbalance, and the adaptation of the batch-normalization statistics. During this period, the convolutional filters were initialized randomly and had to learn both general visual patterns and disease-specific features directly from the dataset. Consequently, the CNN required more epochs than the pretrained model to reach stable performance.

A clear improvement occurred after the learning rate was reduced. At epoch 28, ReduceLROnPlateau decreased the learning rate from 0.001 to approximately 0.0003. A second reduction to approximately 0.00009 occurred at epoch 35. Following these adjustments, the training and validation curves became more stable. The validation accuracy reached 98.90% at epoch 43, while the maximum training accuracy reached 98.47%.

Although the highest validation accuracy appeared at epoch 43, the EarlyStopping callback restored the weights from epoch 45 because that epoch produced the lowest monitored validation loss. This distinction is important because the best epoch depends on the monitored criterion. In the comparative table, epoch 43 is reported as the best epoch based on validation accuracy, while epoch 45 represents the selected epoch based on validation loss.

The difference between the maximum training accuracy and maximum validation accuracy was only 0.43 percentage points. The final testing accuracy was also 98.90%. The close relationship among training, validation, and testing performance indicates that the model achieved a good fit. There was no evidence of severe overfitting in the final epochs because the validation performance remained close to the training performance. The model also did not experience underfitting because both training and testing accuracy substantially exceeded the required 90% threshold.

The low final training and validation losses further indicate that the model developed effective discriminative representations for the ten tomato leaf categories. The combination of Batch Normalization, Dropout, GlobalAveragePooling2D, augmentation, and adaptive learning-rate reduction appears to have provided sufficient regularization while preserving the model's learning capacity.

3.1.2 MobileNetV2 Feature-Extraction Results

The MobileNetV2 model was initially trained using a feature-extraction strategy. During this stage, all layers in the pretrained MobileNetV2 base were frozen, and only the custom classification head was optimized. The classification head contained GlobalAveragePooling2D, a Dense layer with 256 neurons, Batch Normalization, Dropout, and a ten-class softmax output layer.

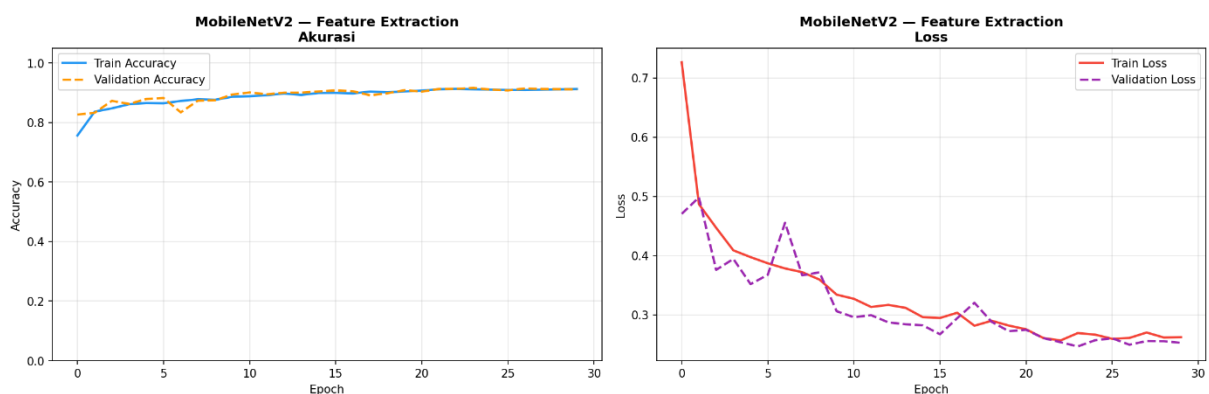


Figure 6. MobileNetV2 feature-extraction accuracy and loss

Figure 6 shows that MobileNetV2 produced relatively high accuracy from the beginning of training. The first-epoch training accuracy reached approximately 75.63%, while the validation accuracy reached 82.64%. This behavior was expected because the convolutional base had already learned general visual features from ImageNet, including edges, shapes, textures, and object components.

During the feature-extraction stage, the training accuracy gradually increased to approximately 91.24%. The highest validation accuracy was 91.60%, first obtained at epoch 24. The training and validation curves remained relatively close throughout the process. This indicates that the frozen pretrained representation generalized adequately and did not produce substantial overfitting.

The training loss declined from approximately 0.73 to 0.26, while the validation loss declined from approximately 0.47 to 0.25. Some fluctuations occurred, particularly in the early epochs, but they were considerably smaller than those observed during the early training of the CNN from scratch. The smoother learning behavior demonstrates one advantage of transfer learning: pretrained weights can provide a stable starting point and accelerate convergence.

However, the feature-extraction accuracy remained below the final performance obtained by the CNN from scratch. The frozen ImageNet features were sufficiently general for tomato leaf classification but were not fully optimized for the fine-grained texture and lesion patterns of the disease classes. Therefore, fine-tuning was applied to adapt the upper layers of MobileNetV2 to the target domain.

3.1.3 MobileNetV2 Fine-Tuning Results

During fine-tuning, the final 30 layers of the MobileNetV2 base were unfrozen. The earlier layers remained frozen because they generally capture transferable low-level features, whereas the later layers represent more task-specific

patterns. A smaller learning rate of (10^{-5}) was used to prevent large parameter updates from destroying the useful pretrained representations.

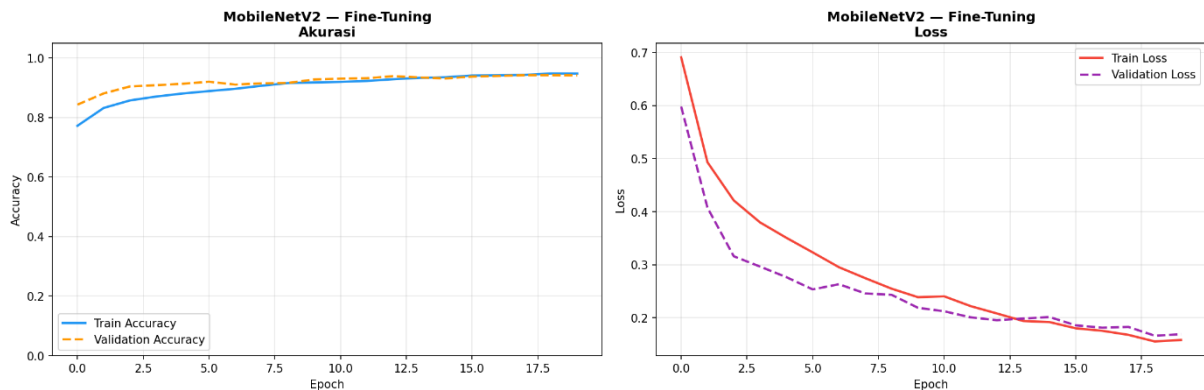


Figure 7. MobileNetV2 fine-tuning accuracy and loss

At the beginning of fine-tuning, the training and validation accuracy temporarily decreased. This drop can also be observed in the combined feature-extraction and fine-tuning history. The decrease was expected because several pretrained layers changed from frozen to trainable, the optimizer was recompiled, and the feature distribution began adapting to the tomato leaf dataset.

After the initial decrease, performance recovered consistently. Training accuracy increased to 94.77%, while validation accuracy reached 94.21%. The maximum validation accuracy was first obtained at fine-tuning epoch 18. The lowest validation loss occurred at epoch 19, and the callback restored the weights from that epoch.

The training loss decreased from approximately 0.69 at the beginning of fine-tuning to 0.16 near the final epoch. Validation loss decreased from approximately 0.59 to 0.17. The two loss curves became closely aligned after approximately 12 epochs, indicating that fine-tuning improved domain adaptation without producing severe overfitting.

The gap between training accuracy and validation accuracy was only 0.56 percentage points. Therefore, the fine-tuned MobileNetV2 model was categorized as a good-fit model. Fine-tuning increased performance beyond the feature-extraction stage, demonstrating that adapting the final pretrained layers was beneficial. Nevertheless, its final accuracy remained below that of the custom CNN.

3.2 Testing Results

Both trained models were evaluated using 1,453 images from the held-out testing subset. These images were not used to update model weights during training. Testing performance was evaluated through loss, accuracy, precision, recall, F1-score, classification reports, and confusion matrices.

3.2.1 CNN From Scratch Testing Results

The CNN from scratch achieved a testing accuracy of 98.90% and a test loss of 0.0348. It correctly classified 1,437 of the 1,453 testing images, producing only 16 incorrect predictions.

Table 2. Classification report of the CNN from scratch

Class	Precision	Recall	F1-Score	Support
Bacterial spot	0.99	0.99	0.99	171
Early blight	1.00	0.93	0.96	80
Healthy	0.98	1.00	0.99	126
Late blight	0.97	0.99	0.98	153
Leaf mold	1.00	0.99	0.99	76
Septoria leaf spot	0.99	1.00	0.99	142
Target spot	0.99	0.99	0.99	112
Tomato mosaic virus	1.00	1.00	1.00	30
Tomato yellow leaf curl virus	1.00	0.99	0.99	429
Two-spotted spider mite	0.99	0.99	0.99	134
Macro average	0.99	0.99	0.99	1,453
Weighted average	0.99	0.99	0.99	1,453

The classification report shows that the model achieved strong precision and recall across all classes. Tomato mosaic virus obtained perfect precision, recall, and F1-score. Healthy leaves and Septoria leaf spot also achieved recall values of 1.00, which means all testing images from these classes were correctly identified.

The lowest recall was obtained for early blight, with a value of 0.93. This indicates that approximately 7% of early blight images were assigned to another class. Even so, early blight precision reached 1.00, meaning that images predicted as early blight were correct.

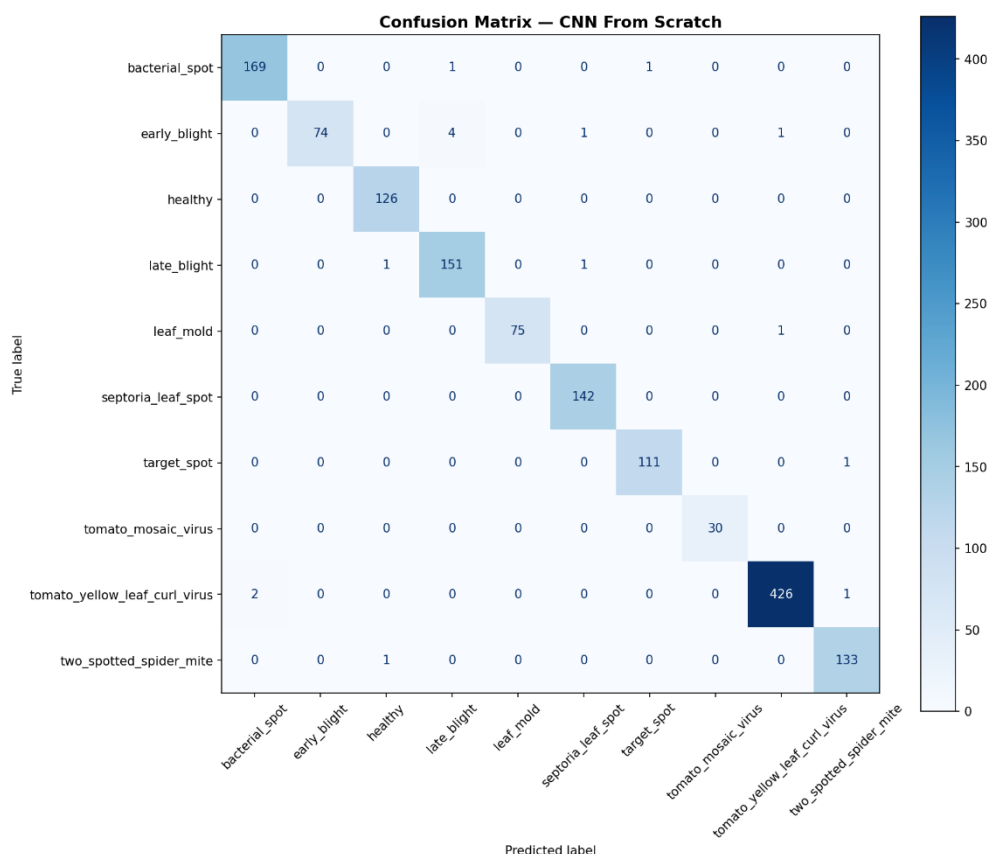


Figure 8. Confusion matrix of the CNN from scratch

The confusion matrix in Figure 8 provides a more detailed view of the 16 errors. Of the 171 bacterial spot images, 169 were classified correctly, while one was predicted as late blight and one as target spot. Among 80 early blight images, 74 were classified correctly; four were classified as late blight, one as Septoria leaf spot, and one as tomato yellow leaf curl virus.

All 126 healthy images were correctly classified. Of the 153 late blight images, 151 were classified correctly, while one was classified as healthy and one as Septoria leaf spot. The model correctly classified 75 of 76 leaf mold images, with one assigned to tomato yellow leaf curl virus. All 142 Septoria leaf spot images and all 30 tomato mosaic virus images were correctly classified.

The model also correctly identified 111 of 112 target spot images, 426 of 429 tomato yellow leaf curl virus images, and 133 of 134 two-spotted spider mite images. Most errors occurred between classes that shared visual characteristics such as brown lesions, yellowing, irregular spots, and damaged tissue. Despite these similarities, the small number of off-diagonal values indicates excellent class separation.

3.2.2 MobileNetV2 Fine-Tuned Testing Results

The fine-tuned MobileNetV2 achieved a testing accuracy of 94.77% and a test loss of 0.1536. It correctly classified 1,377 of the 1,453 test images, resulting in 76 incorrect predictions.

Table 3. Classification report of the fine-tuned MobileNetV2

Class	Precision	Recall	F1-Score	Support
Bacterial spot	0.98	0.96	0.97	171
Early blight	0.91	0.80	0.85	80
Healthy	0.98	0.99	0.98	126
Late blight	0.95	0.95	0.95	153
Leaf mold	0.97	0.86	0.91	76

Septoria leaf spot	0.88	0.93	0.86	142
Target spot	0.81	0.93	0.86	112
Tomato mosaic virus	0.94	1.00	0.97	30
Tomato yellow leaf curl virus	1.00	0.99	0.99	429
Two-spotted spider mite	0.95	0.87	0.91	134
Macro average	0.94	0.93	0.93	1,453
Weighted average	0.95	0.95	0.95	1,453

MobileNetV2 maintained strong performance for healthy leaves, bacterial spot, tomato mosaic virus, and tomato yellow leaf curl virus. However, its performance was lower for several visually similar disease categories. Early blight produced the lowest recall of 0.80, while target spot produced the lowest precision of 0.81. The low precision for target spot indicates that a notable number of images from other categories were incorrectly predicted as target spot. The lower early blight recall indicates that the model failed to detect a larger proportion of actual early blight samples.

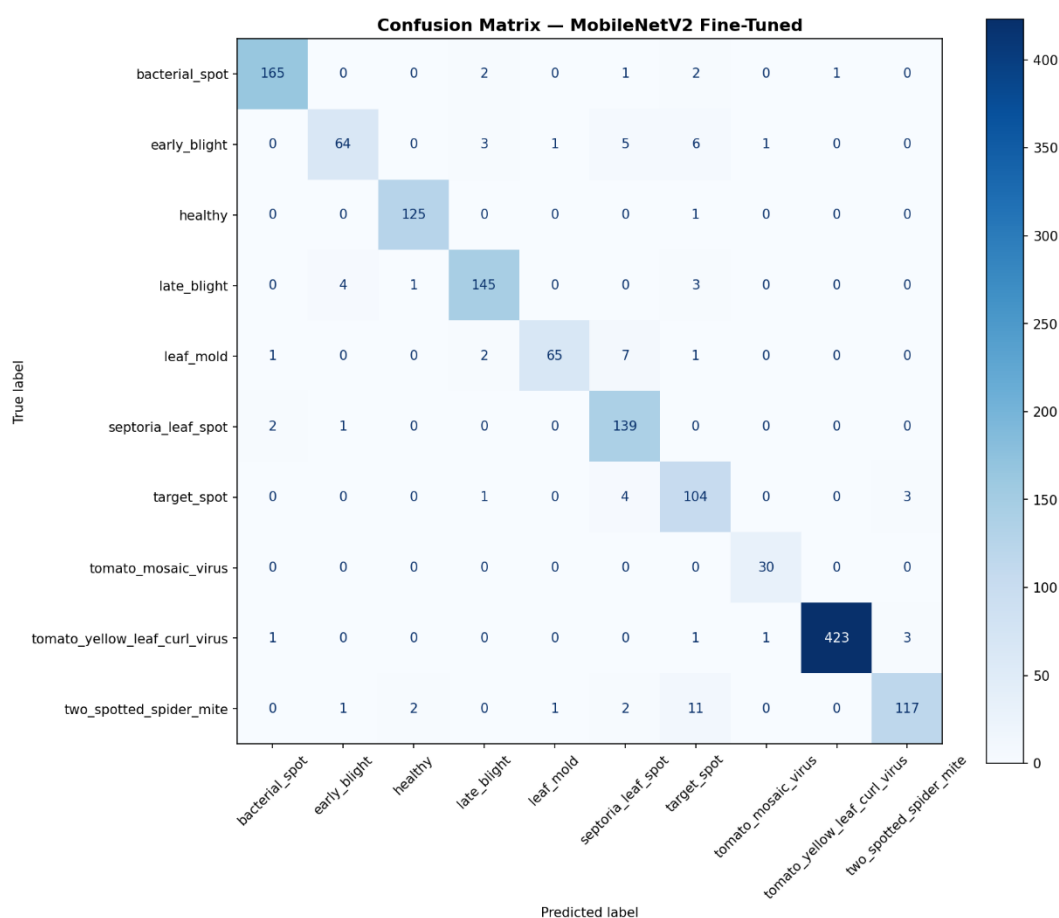


Figure 9. Confusion matrix of the fine-tuned MobileNetV2

The MobileNetV2 confusion matrix shows that 165 of 171 bacterial spot images were correctly classified. Early blight produced 64 correct predictions from 80 images. The remaining images were primarily confused with target spot, Septoria leaf spot, and late blight.

The model correctly classified 125 of 126 healthy images and 145 of 153 late blight images. Leaf mold showed 65 correct predictions from 76 testing images, with seven images classified as Septoria leaf spot. Target spot obtained 104 correct predictions from 112 images, while four were classified as Septoria leaf spot and three as two-spotted spider mite.

The most substantial confusion for two-spotted spider mite occurred with target spot. Of the 134 images, 117 were classified correctly, while 11 were predicted as target spot. These errors suggest that MobileNetV2 had greater difficulty distinguishing classes characterized by comparable spotting, discoloration, and texture patterns.

3.3 Comparative Analysis

Table 2. Classification report of the CNN from scratch

Parameter	CNN From Scratch	MobileNetV2 Fine-Tuned
Total parameters	852,522	2,589,514
Trainable parameters	850,090	1,857,418 after fine-tuning
Best epoch by validation accuracy	43	18 during fine-tuning
Best training accuracy	98.47%	94.77%
Best validation accuracy	98.90%	94.21%
Testing accuracy	98.90%	94.77%
Test loss	0.0348	0.1536
Macro precision	0.99	0.94
Macro recall	0.99	0.93
Macro F1-score	0.99	0.93
Approximate training time	135.8 minutes	117.1 minutes
Model-size information	Measure .keras file before submission	SavedModel $\approx$ 21.46 MB; TFLite 7.17 MB; TFJS $\approx$ 9.83 MB

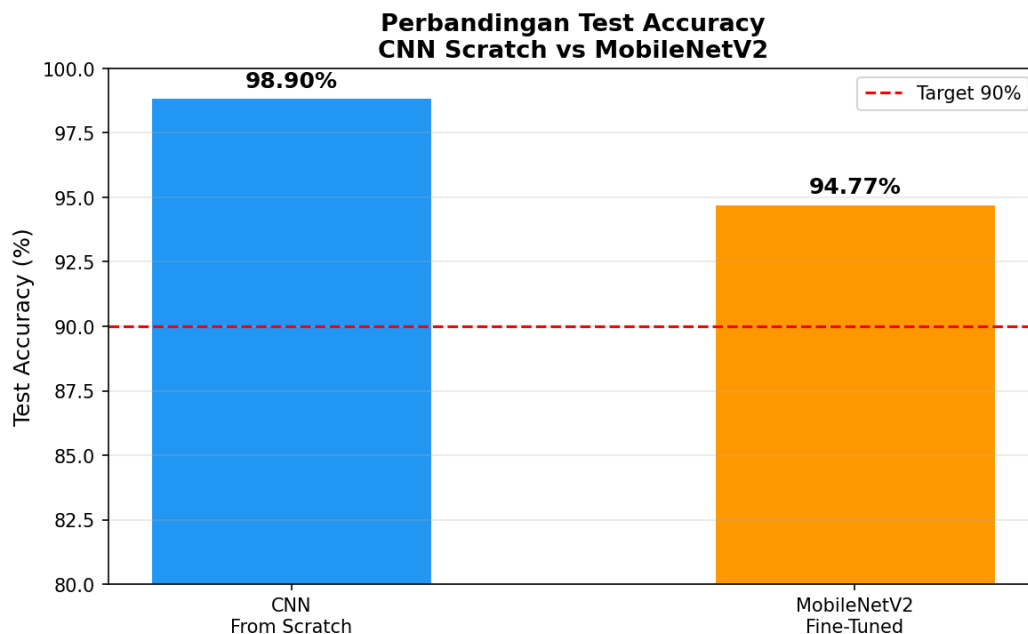


Figure 10. Testing accuracy comparison between the two models

The CNN from scratch outperformed MobileNetV2 by 4.13 percentage points in testing accuracy. It also achieved a test loss approximately 0.119 lower than MobileNetV2. These differences show that transfer learning did not automatically produce the highest performance in this experiment.

The custom CNN contained approximately one-third of the total parameters of MobileNetV2. MobileNetV2 had approximately 3.04 times more total parameters than the CNN. After the final 30 layers were unfrozen, MobileNetV2 also had more than twice as many trainable parameters as the CNN. Therefore, the performance advantage of the CNN cannot be attributed to greater parameter complexity.

The superior CNN result may be explained by the characteristics of the PlantVillage dataset. The dataset contained more than 14,000 domain-specific tomato leaf images with relatively controlled backgrounds. This amount of data was sufficient for the custom model to learn disease-specific textures, lesion patterns, color changes, and leaf structures directly. By contrast, MobileNetV2 was initialized using general ImageNet features that were not specifically learned from plant disease symptoms.

Transfer learning nevertheless provided advantages. The feature-extraction stage converged faster and began with considerably higher accuracy. MobileNetV2 also completed feature extraction and fine-tuning in approximately 117.1 minutes, compared with approximately 135.8 minutes for the CNN. This represents a reduction of approximately 18.7 minutes under the same execution environment.

The MobileNetV2 TFLite model was only 2.71 MB after conversion and lightweight optimization. Consequently, MobileNetV2 remains attractive for mobile and embedded deployment even though its classification performance was lower. The CNN is preferable when maximum accuracy on the current dataset is the main requirement.

MobileNetV2 may be preferable when compact deployment, faster convergence, and efficient inference are given greater priority.

3.4 Model Deployment and Inference

The trained MobileNetV2 model was exported into three deployment formats: TensorFlow SavedModel, TensorFlow Lite, and TensorFlow.js. SavedModel was generated using the Keras export function and contained the model computation graph, signature, and trained variables. Its stored components occupied approximately 21.46 MB.

The TensorFlow Lite model was generated from SavedModel using the TFLite converter with default optimization. The resulting .tflite file had a size of approximately 2.71 MB. This substantial reduction makes the model suitable for Android, mobile, edge, or embedded applications.

TensorFlow.js conversion generated a model.json file and three binary weight shards with a combined size of approximately 9.83 MB. This format enables inference in a browser using JavaScript without requiring the image to be uploaded to an external server. A separate class_names.json file was also stored to preserve the mapping between output indexes and disease labels.

Inference was first tested using ten held-out testing images, with one image selected from each class. The displayed results included the actual label, predicted label, and prediction probability.

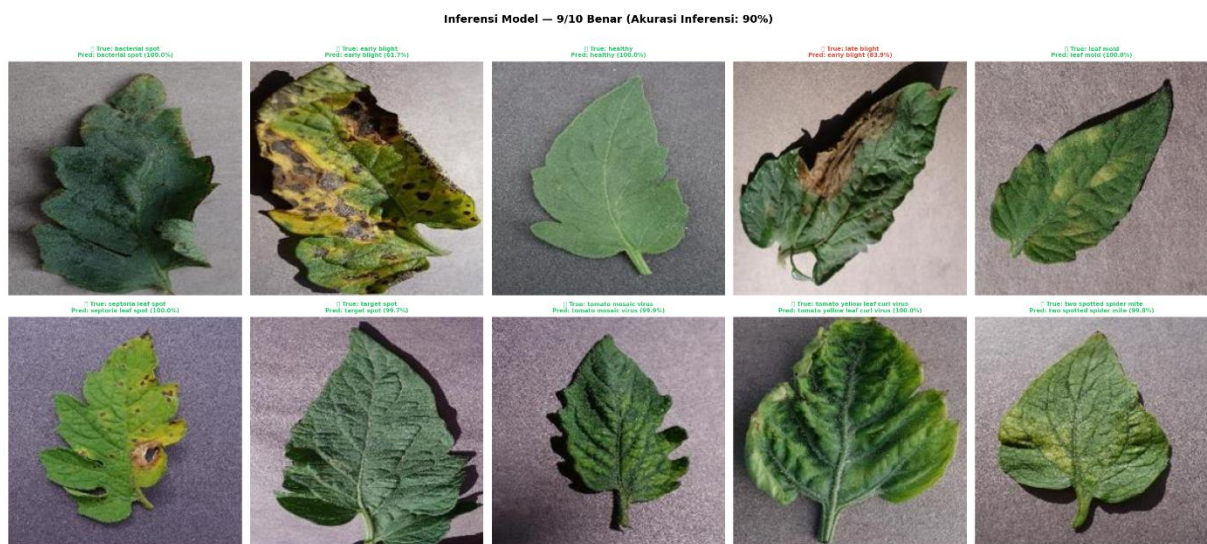


Figure 11. Inference results on ten previously unseen testing images

Nine of the ten images were correctly classified, corresponding to a sample inference accuracy of 90%. Bacterial spot was correctly predicted with 99.98% probability. Early blight was correctly predicted with 61.73%, while the healthy class reached 99.99%. Leaf mold and Septoria leaf spot were predicted with probabilities of 100%.

Target spot, tomato mosaic virus, tomato yellow leaf curl virus, and two-spotted spider mite were also correctly classified with probabilities above 99%. The only incorrect prediction occurred for a late blight image, which was classified as early blight with 83.90% probability. This error is consistent with the confusion-matrix analysis because late blight and early blight can exhibit related brown lesions, damaged areas, and yellowing patterns.

Inference was additionally conducted using the TensorFlow Lite model on five images. Four of the five images were correctly classified. The late blight image was again predicted as early blight, with a probability of 80.05%. The consistency of this error suggests that it was primarily caused by visual similarity rather than by the model-conversion process.

The deployment experiment demonstrates that the trained network can be converted and executed outside the original Keras training environment. However, the inference images were selected from the held-out PlantVillage testing subset. Therefore, the results represent internal test generalization rather than full real-world field validation. Future evaluation should use independently captured tomato leaf images with natural backgrounds, uneven illumination, occlusion, multiple leaves, and varied camera distances.

4. CONCLUSION

This study successfully developed and compared a Convolutional Neural Network from scratch and a fine-tuned MobileNetV2 model for multiclass tomato leaf disease classification. Both models were trained and evaluated using 14,521 PlantVillage tomato leaf images distributed across 10 classes and divided into training, validation, and testing subsets using an 80:10:10 ratio. The evaluation results demonstrated that both approaches exceeded the required testing accuracy of 90%, indicating that deep learning can effectively distinguish healthy tomato leaves from several disease and pest categories.

The CNN from scratch achieved the best overall performance, with a testing accuracy of 98.90%, a test loss of 0.0348, and macro precision, recall, and F1-score values of approximately 0.99. In comparison, the fine-tuned MobileNetV2 achieved a testing accuracy of 94.77%, a test loss of 0.1536, macro precision of 0.94, and macro recall and F1-score of 0.93. The confusion-matrix analysis also showed that the CNN from scratch produced fewer classification errors, particularly among visually similar disease classes. Therefore, the objective of determining the most effective model for the selected dataset was achieved, and the CNN from scratch was identified as the best-performing model.

These findings also demonstrate that transfer learning does not always produce higher accuracy than a custom CNN. A sufficiently large and visually consistent domain-specific dataset may allow a model trained from random initialization to learn highly relevant disease features. Nevertheless, MobileNetV2 provided faster convergence and produced a compact TensorFlow Lite model, making it a practical option for mobile and embedded deployment.

Future research should evaluate both models using independently captured field images containing natural backgrounds, uneven illumination, occlusion, multiple leaves, and different camera distances. Additional improvements may include class-balancing techniques, more realistic data augmentation, MobileNetV2-specific preprocessing, hyperparameter optimization, model quantization, and explainability methods such as Grad-CAM. The selected model should also be integrated into a mobile or web-based diagnostic application to assess its real-time performance and practical usefulness for tomato disease identification.

REFERENCES

- [1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Front. Plant Sci.*, vol. 7, p. 1419, 2016, doi: 10.3389/fpls.2016.01419.
- [2] S. Ahmed, Md. B. Hasan, T. Ahmed, R. K. Sony, and Md. H. Kabir, "Less is More: Lighter and Faster Deep Neural Architecture for Tomato Leaf Disease Classification." 2021. doi: 10.48550/arXiv.2109.02394.
- [3] A. Sharma, B. Rajesh, and M. Javed, "Detection of Plant Leaf Disease Directly in the JPEG Compressed Domain Using Transfer Learning Technique." 2021. doi: 10.48550/arXiv.2107.04813.
- [4] P. S. Thakur, P. Khanna, T. Sheorey, and A. Ojha, "Explainable Vision Transformer Enabled Convolutional Neural Network for Plant Disease Identification: PlantXViT." 2022. doi: 10.48550/arXiv.2207.07919.
- [5] S. Thuseethan, P. Vigneshwaran, J. Charles, and C. Wimalasooriya, "Siamese Network-Based Lightweight Framework for Tomato Leaf Disease Recognition." 2022. doi: 10.48550/arXiv.2209.11214.
- [6] Md. H. Ashmafee, T. Ahmed, S. Ahmed, Md. B. Hasan, M. N. Jahan, and A. B. M. A. Rahman, "An Efficient Transfer Learning-Based Approach for Apple Leaf Disease Classification." 2023. doi: 10.48550/arXiv.2304.06520.
- [7] J. Yao, S. N. Tran, S. Garg, and S. Sawyer, "Deep Learning for Plant Identification and Disease Classification from Leaf Images: Multi-Prediction Approaches." 2023. doi: 10.48550/arXiv.2310.16273.
- [8] A. Yasin and R. Fatima, "On the Image-Based Detection of Tomato and Corn Leaves Diseases: An In-Depth Comparative Experiment." 2023. doi: 10.48550/arXiv.2312.08659.
- [9] S. Jha, V. Luhach, G. S. Gupta, and B. Singh, "Crop Disease Classification Using Support Vector Machines with Green Chromatic Coordinate and Attention-Based Feature Extraction for IoT-Based Smart Agricultural Applications." 2023. doi: 10.48550/arXiv.2311.00429.
- [10] J. Amara, B. König-Ries, and S. Samuel, "Concept Explainability for Plant Diseases Classification." 2023. doi: 10.48550/arXiv.2309.08739.

- [11] U. Y. Tambe, A. Shobanadevi, A. Shanthini, and H.-C. Hsu, "Potato Leaf Disease Classification Using Deep Learning: A Convolutional Neural Network Approach." 2023. doi: 10.48550/arXiv.2311.02338.
- [12] A. Ouamane *et al.*, "Enhancing Plant Disease Detection: A Novel CNN-Based Approach with Tensor Subspace Learning and HOWSVD-MD." 2024. doi: 10.48550/arXiv.2405.20058.
- [13] J. Amara, B. König-Ries, and S. Samuel, "Explainability of Deep Learning-Based Plant Disease Classifiers Through Automated Concept Identification." 2024. doi: 10.48550/arXiv.2412.07408.
- [14] Y. Ferdi, "Data Augmentation Through Background Removal for Apple Leaf Disease Classification Using the MobileNetV2 Model." 2024. doi: 10.48550/arXiv.2412.01854.
- [15] M. M. Hosen and Md. H. Islam, "Aggrotech: Leveraging Deep Learning for Sustainable Tomato Disease Management." 2025. doi: 10.48550/arXiv.2501.12052.
- [16] S. Mehnaz and Md. T. Islam, "Rice Leaf Disease Detection: A Comparative Study Between CNN, Transformer and Non-Neural Network Architectures." 2025. doi: 10.48550/arXiv.2501.06740.
- [17] S. Kanakala and S. Ningappa, "Detection and Classification of Diseases in Multi-Crop Leaves Using LSTM and CNN Models." 2025. doi: 10.48550/arXiv.2505.00741.
- [18] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520. doi: 10.1109/CVPR.2018.00474.